

Игра в мозаику

Барчин и Чарос играют в игру на сетке из $2N \times 2M$ клеток. Строки пронумерованы от 0 до $2N - 1$ сверху вниз, а столбцы — от 0 до $2M - 1$ слева направо. Для $0 \leq i < 2N$ и $0 \leq j < 2M$ мы обозначаем клетку в строке i и столбце j как (i, j) .

Барчин выдает Чарос $N \cdot M$ **блоков** один за другим. Каждый блок представляет собой квадрат 2×2 , состоящий из четырех **плиток** 1×1 . Каждая плитка покрашена либо в черный, либо в белый цвет, при этом гарантируется, что **хотя бы одна плитка будет белой**.

Чарос должна разместить каждый блок на сетке **сразу после его получения**, не зная, какие блоки она получит позже. Блоки нельзя вращать. Каждый блок должен быть полностью помещен внутри сетки, покрывая ровно четыре клетки сетки. Кроме того, верхняя левая плитка каждого блока должна покрывать клетку, координаты строки и столбца которой четные. Каждая клетка сетки может быть покрыта не более чем одним блоком.

Барчин выигрывает игру, если после размещения очередного блока образуется квадрат из клеток размером 2×2 , покрытый четырьмя черными плитками. Формально, если клетки (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ покрыты черными плитками для некоторого $0 \leq a < 2N - 1$ и $0 \leq b < 2M - 1$, то Барчин выигрывает. Индексы a и b не обязательно должны быть четными.

Чарос победит, если расставит все блоки $N \cdot M$, не допустив того, чтобы в процессе игры Барчин выиграла. Обратите внимание, что размещенные $N \cdot M$ блоков полностью покроют сетку.

Ваша задача — разработать стратегию, которая позволит Чарос выиграть игру. Можно доказать, что при заданных ограничениях Чарос всегда может расставить блоки таким образом, чтобы гарантировать победу независимо от раскраски блоков, которые она получит позже.

Детали реализации

Вам следует реализовать две функции:

```
void init(int N, int M)
```

- N : половина количества строк в сетке.

- M : половина количества столбцов в сетке.
- Функция вызывается ровно один раз для каждого теста, в начале выполнения вашей программы.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : цвета верхнего левого, верхнего правого, нижнего левого и нижнего правого элементов текущего блока соответственно, как показано на рисунке ниже. Каждое значение равно либо 0 (белый), либо 1 (черный).
- Эта функция вызывается ровно $N \cdot M$ раз для каждого теста после первоначального вызова `init`.



Эта функция должна возвращать пару целых чисел (i, j) , где i — это координаты строки, а j — координаты столбца клетки, в которую следует поместить верхнюю левую плитку этого блока. Значения i и j должны быть четными, а область 2×2 , покрываемая блоком, не должна перекрывать ни один из ранее размещенных блоков.

Если `receive_block` вернет пару, не удовлетворяющую этим требованиям, или если после размещения блока появится квадрат из клеток размером 2×2 , полностью покрытый черными плитками, грейдер немедленно завершит вашу программу, и вердикт по тесту будет `Output isn't correct`.

Поведение грейдера **не является адаптивным**. Это означает, что последовательность блоков, которые Барчин передает Чарос, фиксируется до вызова функции `init`.

Ограничения

Пусть S — это количество черных плиток среди четырех плиток отдельного блока. То есть $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ для каждого блока.

Подзадачи

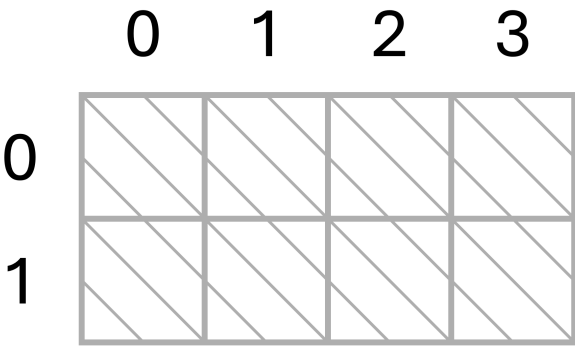
Подзадача	Оценка	Дополнительные ограничения
1	6	$S = 1$ для каждого блока, и $N = 2$.
2	16	$S = 3$ для каждого блока. $N = M$, N — четное число, и каждая из четырех возможных раскрасок блоков встречается ровно $\frac{N^2}{4}$ раз.
3	10	$S = 1$ для каждого блока.
4	29	$S \leq 2$ для каждого блока.
5	39	Без дополнительных ограничений.

Пример

Рассмотрим игру, в которой $N = 1$ и $M = 2$, так что сетка имеет 2 строки и 4 столбца. Сначала грейдер вызывает:

```
init(1, 2)
```

Изначально все клетки сетки пусты. Сетка выглядит следующим образом:



Всего нужно разместить $N \cdot M = 2$ блока. Предположим, Барчин выдает блок с тремя черными и одной белой клеткой в верхнем правом углу. Грейдер вызывает:

```
receive_block(1, 0, 1, 1)
```

Чарос решает разместить этот блок в левой части сетки, вернув $(0, 0)$.

Теперь сетка выглядит так:

	0	1	2	3
0	black	white	diagonal	diagonal
1	black	black	diagonal	diagonal

Затем Барчин выдает еще один блок с тремя черными и одной белой плиткой в верхнем левом углу:

```
receive_block(0, 1, 1, 1)
```

Единственная оставшаяся клетка с четными координатами строки и столбца, которая может служить верхним левым углом блока размером 2×2 — это $(0, 2)$, поэтому Чарос возвращает $(0, 2)$. В итоге сетка выглядит так:

	0	1	2	3
0	black	white	white	black
1	black	black	black	black

Ни один квадрат 2×2 не покрыт полностью черными плитками, поэтому Чарос успешно разместила все блоки, не позволив Барчин выиграть. Чарос выигрывает игру.

Пример грейдера

Формат ввода:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Формат вывода:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Здесь $R[k]$ и $C[k]$ — это пара целых чисел, возвращаемых k -м вызовом функции `receive_block`.